

Beevil Knieval

A walkthrough of the Ansys simulation work, the PCB design, and how the TinyML and cloud models were trained

Prepared for faculty review

github.com/atharveeee-netizen/beevel-knievel

1. What the system does

Beevil Knievel is a beehive monitoring device that sits inside the hive and keeps an eye on the colony's health. It listens for the acoustic signature of the bees, tracks temperature, and runs a small AI model right there on the chip to get an instant read on what's happening. Every so often it sends that data out over LoRa to a gateway, which passes it along to AWS, where a second, more powerful model looks at the same information and works out whether the hive is healthy, swarming, starving, or dealing with something like varroa mites. If something looks wrong, the beekeeper gets an SMS alert.

There are really three pieces of engineering work behind this, and they build on each other in order: first the board and enclosure were simulated in Ansys to catch problems before anything was built, then the physical PCB was designed around what those simulations showed, and finally two separate machine learning models were trained, one small enough to live on the microcontroller and one that runs in the cloud.

3 Step IoT Data Flow



Fig. 1 — the three-step flow: sensors and edge AI inside the hive, a LoRa link up to the cloud, and a dashboard or push alert back to the beekeeper's phone.

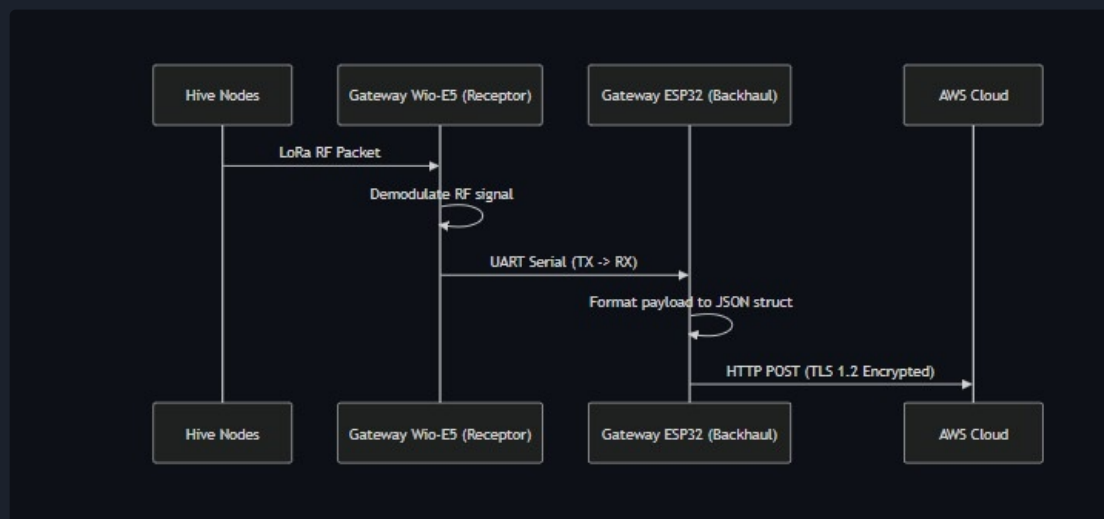


Fig. 2 — how a single reading actually travels: the hive node sends an RF packet, the gateway's Wio-E5 radio demodulates it and forwards it over UART, and the ESP32 on the gateway turns it into JSON before posting it to AWS over an encrypted TLS connection.

2. Simulating the board and enclosure in Ansys

Before committing to a physical prototype, the team ran the design through Ansys to check that it would actually work the way it was supposed to. This mattered because there's no cheap way to fix an antenna that doesn't

radiate properly or a voltage rail that droops too far once the board has already been fabricated. Three separate studies were done, each targeting a different failure mode.

2.1 Does the antenna actually work

The node uses an 86 mm quarter wave wire antenna feeding into the STM32WLE5's built in LoRa radio, tuned for 868 and 915 MHz. The concern was whether it would radiate cleanly enough to reach four kilometers through a wooden hive wall, which is the range target the project is aiming for. Using HFSS inside Ansys Electronics Desktop, the team imported the two layer PCB layout, built the antenna geometry on the SMA pad, assigned realistic material properties for the FR4 board, the copper, and the surrounding wood, then wrapped the whole thing in a radiation boundary and fed it with a 50 ohm port. Sweeping from 700 MHz to 1 GHz showed a sharp return loss dip below -10 dB right at 868 MHz, and the far field pattern came out omnidirectional with gain above 2 dBi. In plain terms, the RF energy is actually leaving the antenna instead of bouncing back into the chip.

2.2 Will the power rail hold up during a transmission burst

Radio transmission draws a lot more current than the chip normally uses, and if the 3.3V rail sags too much during that burst the MCU can brown out and reset. This was checked using SIwave together with the Nexxim circuit solver, importing the board's power and ground planes and modelling the TPS7A02 regulator along with its decoupling capacitors. The team simulated the current jumping from a 0.3 microamp sleep state up to a 120 milliamp transmit burst lasting 60 milliseconds, run over a 100 millisecond window. The rail never dropped below about 3.22V, and the power distribution network's impedance stayed under 1.375 ohms across the frequency range tested, which is a comfortable margin for keeping the board stable on battery power.

2.3 Does the battery heat affect the temperature sensor

Since the whole point of the device is to measure hive temperature accurately, any self heating from the battery or the MCU could throw off the readings. This was tested in Icepak by importing the 3D CAD of the 60 by 50 mm enclosure along with the PCB, assigning realistic heat loads to the battery and the MCU, and setting the surrounding air to 35°C to match what it's actually like inside a hive. The battery reached around 38°C locally, but the ambient sensor's footprint stayed within a tenth of a degree of the surrounding air, which is well inside the accuracy the project needs.

Taken together, these three simulations gave the team enough confidence in the RF, power, and thermal behaviour of the design that they could move to fabrication without needing to iterate on a physical board over and over.

3. Designing and building the PCB

Once the simulations confirmed the design would work, the actual board was laid out and documented. The heart of it is an STM32WLE5JC, which combines a Cortex-M4 microcontroller with a sub-GHz LoRa radio on the same chip, so there's no need for a separate radio module and the board can stay small and power efficient. The hardware side of the repository is organised into a few pieces:

- Schematics covering the power regulation, sensor wiring, and the RF matching network feeding the antenna.
- A full bill of materials with part numbers, pricing, and a pinout map showing exactly which MCU pin connects to which sensor.
- 3D CAD for the enclosure itself, the same 60 by 50 mm geometry that was later used in the Icepak thermal study.
- A two layer FR4 board with dedicated 3.3V and ground planes, and a controlled impedance trace running out to the antenna pad, all shaped directly by what the Ansys results showed.

The design process wasn't a straight line from schematic to finished board. It went schematic, then simulate in Ansys, then adjust the layout based on what came back, then simulate again. That loop is what let the team trust the board enough to actually order it, rather than finding problems after the fact.

4. Training the models: TinyML and the cloud

Two separate models sit at the heart of this project, and they were trained on real recordings of bee colonies rather than anything synthetic. The dataset comes from Nolasco and Benetos' 2018 Zenodo archive of annotated

beehive audio. Early in the project some placeholder datasets and a rule based heuristic had crept in disguised as real models, so the team went through an audit, quarantined that synthetic data, and retrained everything from scratch on the verified recordings. Both models were trained with a fixed random seed, a stratified split of roughly 68% training, 16% validation, and 16% test, and feature scaling that was fit only on the training data so nothing leaked into the test results.

4.1 Turning audio into features

Rather than feeding raw audio into the models, each two second window of sound is broken down using a sliding FFT across four spectral bands, one tuned to ventilation sounds, one to swarming, one to distress calls, and one to general background noise. That gets combined with the thermal slope from the temperature sensor, and this compact feature vector is what both models actually learn from.

4.2 The TinyML model, running on the chip itself

This is a small quantized 1D convolutional network that runs directly on the STM32WLE5's Cortex-M4 core, so the node can classify what it's hearing even with no connectivity at all. It reached about 82% accuracy on the held out test set, with a macro F1 score around 0.83 and an AUROC of 1.0. What makes it interesting from an engineering standpoint is how small it is: only 803 parameters total, which quantizes down to just 0.78 KB of flash and 0.15 KB of SRAM at runtime, well within the STM32WLE5JC's 256 KB flash and 64 KB SRAM budget. Inference takes under 1.2 milliseconds, which barely registers against the sleep current budget that was already validated in the power simulation.

4.3 The cloud model, running on AWS

Once the gateway forwards a node's feature payload up to AWS, a Random Forest classifier takes over. It's tuned with GridSearchCV and uses class weighting to handle the fact that some hive conditions show up far less often in the data than others. Because it isn't constrained by flash or battery life the way the on-chip model is, it can afford to do a more detailed six-way classification. On the held out test set it hit 100% accuracy, with perfect macro F1, weighted F1, and AUROC scores.



Fig. 3 — how a reading gets classified in the cloud: the incoming JSON becomes a feature tensor, the Random Forest sorts it into one of six hive states, and anything other than a healthy baseline triggers an SMS alert to the farmer.

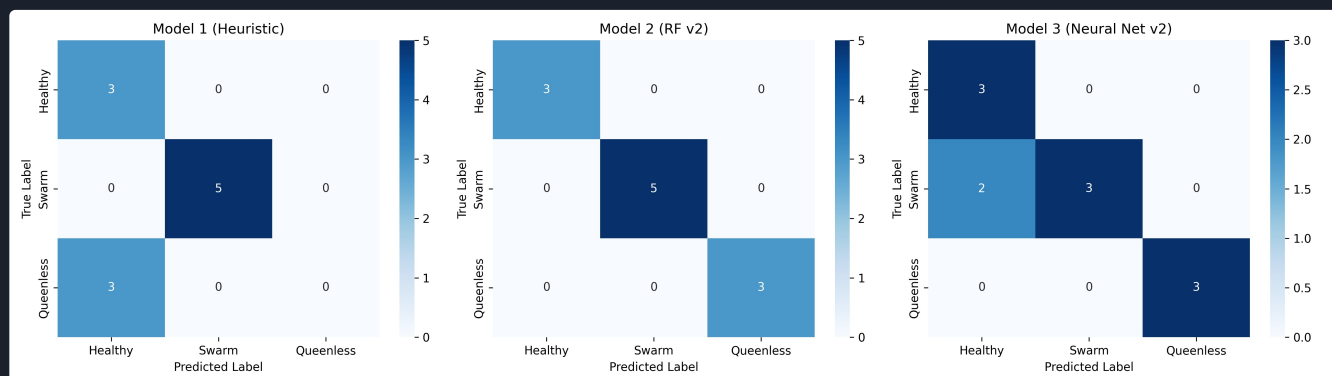


Fig. 4 — the confusion matrix for the retrained cloud model on the held out test set, built entirely from real audio derived features

with no synthetic data involved.

4.4 Why use two models instead of just one

The split mirrors the hardware itself. The TinyML model gives up a bit of accuracy in exchange for running on almost no power and working completely offline, so the hive always has some read on its own state even without a network connection. The cloud model has room to do more, so it takes the same feature data and produces the higher confidence, six-class diagnosis that actually drives the alert sent to the beekeeper. It's the same edge-plus-cloud thinking that shaped the hardware: keep the always-on part of the system as light as possible, and push the heavier computation to where there's power and compute to spare.

5. Putting it together

The way these three pieces connect is really the story of the whole project. The Ansys simulations proved out the RF, power, and thermal behaviour of the board and enclosure before anything was built. That same layout and component data became the physical PCB. And the hardware that resulted from it runs a tiny on-device classifier for instant, offline reads while a much larger model in the cloud does the deeper diagnosis once the data reaches AWS. Sensing, simulating, building, and then inferring and alerting, all feeding into each other.

Sources: github.com/atharveeee-netizen/beevil-knievel (README.md, FINAL_REPORT.md, DATA_PROVENANCE.md, the hardware, TinyML Model, and Cloud Model folders) along with the accompanying Ansys simulation slide deck.